

Министерство образования Республики Беларусь

Учреждение образования

**БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ**

Факультет компьютерных систем и сетей

Кафедра информатики

Дисциплина «Операционные среды и системное программирование»

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА

к курсовому проекту
на тему

РЕАЛИЗАЦИЯ ПРОСТОГО DNS-СЕРВЕРА

БГУИР КП 6-05-0612-02 022 ПЗ

Студент

Д. Ю. Себелев

Руководитель

Н. Ю. Гриценко

Минск 2026

СОДЕРЖАНИЕ

Введение	5
1 Архитектура программного обеспечения	6
1.1 Структура сетевого взаимодействия на базе протокола UDP	6
1.2 Иерархия DNS-серверов и процесс разрешения запросов	7
1.3 Формат файла зоны	9
1.4 Организация кэша в памяти	10
2 Платформа программного обеспечения	11
2.1 Язык программирования	11
2.2 Операционная система	11
2.3 Принципы сетевого программирования на базе сокетов	12
2.4 Работа с бинарными структурами данных и сериализация	13
3 Теоретическое обоснование разработки программного продукта	15
3.1 Структура DNS-пакета	15
3.2 Типы ресурсных записей	18
3.3 Принципы кэширования DNS-записей	19
4 Проектирование функциональных возможностей программы	21
4.1 Алгоритм разбора входящего DNS-запроса	21
4.2 Двухуровневая система поиска ресурсных записей	22
4.3 Формирование DNS-ответа	23
4.4 Обработка несуществующих доменов и негативное кэширование . . .	24
4.5 Многопоточная архитектура сервера	25
4.6 Логирование и диагностика	28
5 Архитектура разрабатываемой программы	32
5.1 Функциональная схема обработки запросов и кэширования	32
5.2 Описание алгоритма работы сервера	33
Заключение	34
Список использованных источников	35
Приложение А (Обязательное) Листинг программного кода	36
Приложение Б (Обязательное) Функциональная схема обработки запросов и кэширования	46
Приложение В (Обязательное) Блок-схема алгоритма поиска и формирования DNS-пакета ответа	47
Приложение Г (Обязательное) Графический интерфейс пользователя	48
Приложение Д (Обязательное) Ведомость курсового проекта	49

ВВЕДЕНИЕ

Система доменных имён (*DNS*) является одним из фундаментальных компонентов инфраструктуры сети Интернет. Каждый раз при обращении к веб-сайту, отправке электронного письма или использовании любого сетевого сервиса выполняется *DNS*-запрос, преобразующий доменное имя в *IP*-адрес. По данным крупных *DNS*-операторов, суммарное число запросов в глобальной сети исчисляется сотнями миллиардов в сутки.

Понимание внутреннего устройства *DNS*-сервера имеет практическую ценность как для системных программистов, так и для специалистов по сетевой безопасности. Реализация собственного сервера имён позволяет глубоко изучить бинарный формат *DNS*-пакетов, механизмы кэширования, управление параллельными потоками и работу с сокетами на уровне операционной системы.

Локальные *DNS*-серверы широко применяются в корпоративных интрасетях, средах разработки и тестирования, а также в контейнерных платформах. Так, в инфраструктурах на базе *Docker* и *Kubernetes* каждый кластер использует внутренний *DNS* для разрешения имён сервисов без обращения к публичным серверам. Возможность развернуть собственный сервер имён позволяет снизить зависимость от сторонней инфраструктуры, обеспечить предсказуемое время отклика и реализовать нестандартные политики разрешения имён.

Техническая сложность реализации *DNS* обусловлена рядом факторов: специфическим бинарным форматом пакетов с механизмом сжатия данных, необходимостью корректной обработки параллельных запросов без состояния гонки в кэше, а также требованием корректного завершения всех потоков при остановке сервера.

Целью данного курсового проекта является реализация *DNS*-сервера для локальной зоны, принимающего *UDP*-запросы, выполняющего парсинг *DNS*-пакетов и возвращающего ответы для записей типов *A*, *AAAA* и *CNAME* с поддержкой многопоточной обработки и кэширования.

Для достижения поставленной цели необходимо решить следующие задачи:

- 1 Изучить структуру протокола *DNS*, формат бинарных пакетов и способы кодирования доменных имён.
- 2 Разработать парсер входящих *DNS*-запросов и формирователь ответных пакетов.
- 3 Реализовать загрузку и хранение ресурсных записей локальной зоны (*A*, *AAAA*, *CNAME*) из файла зоны.
- 4 Реализовать многопоточную обработку входящих *UDP*-запросов на основе пула потоков.
- 5 Реализовать кэширование ответов с учётом значения *TTL* и обеспечить ведение консольного журнала событий.

1 АРХИТЕКТУРА ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ

В данном разделе рассматриваются ключевые архитектурные решения, принятые при проектировании *DNS*-сервера: выбор транспортного протокола, организация хранения данных зоны и кэширования.

1.1 Структура сетевого взаимодействия на базе протокола UDP

Задача построения высокопроизводительного сетевого сервиса начинается с выбора транспортного протокола. Современный стек сетевых протоколов предлагает два принципиально разных подхода к доставке данных между узлами сети: протокол с установлением соединения (*TCP*) и протокол передачи датаграмм (*UDP*).

Протокол *TCP*, прежде чем начать передачу полезных данных, выполняет процедуру согласования между отправителем и получателем. Эта процедура гарантирует, что обе стороны готовы к обмену, создаёт виртуальный канал связи и обеспечивает надёжную упорядоченную доставку каждого фрагмента данных. Цена такой надёжности – дополнительные сетевые обмены на установление и завершение соединения, а также накладные расходы на поддержание его состояния в памяти сервера для каждого подключённого клиента.

Протокол *UDP* лишён этих гарантий: каждый пакет является самостоятельным и независимым. Отправитель не знает, доставлен ли пакет, а получатель не обязан принимать пакеты в том же порядке, в котором они были отправлены. Казалось бы, это делает *UDP* непригодным для серьёзных задач. Однако именно отсутствие накладных расходов делает его идеальным для сервисов, работающих по модели «запрос – ответ». На уровне прикладной логики *DNS* каждое взаимодействие представляет собой пару пакетов: запрос от клиента и ответ от сервера. *UDP* не требует ничего сверх этого, тогда как *TCP* дополнительно генерирует служебный трафик в обоих направлениях – подтверждения, сегменты установления и завершения соединения, – необходимый для поддержания состояния протокола. Потеря единственного запроса при использовании *UDP* решается тривиальным повторным обращением на прикладном уровне.

Структура *UDP*-пакета предельно проста и содержит минимальный набор полей, необходимых для доставки данных. Заголовок состоит из четырёх полей фиксированной длины: порт источника (2 байта), порт назначения (2 байта), длина пакета (2 байта) и контрольная сумма (2 байта). Общий размер заголовка составляет 8 байт, что делает *UDP* одним из наиболее эффективных протоколов транспортного уровня. Структура заголовка *UDP*-пакета представлена на рисунке 1.1. Поле порта источника позволяет получателю отправить ответ, поле порта назначения указывает, какому приложению предназначены данные, поле длины содержит общий размер пакета, включая данные, а контрольная сумма обеспечивает базовую проверку целостности. За заголовком следует полезная нагрузка произвольной длины, содержимое которой интерпретируется на прикладном уровне.

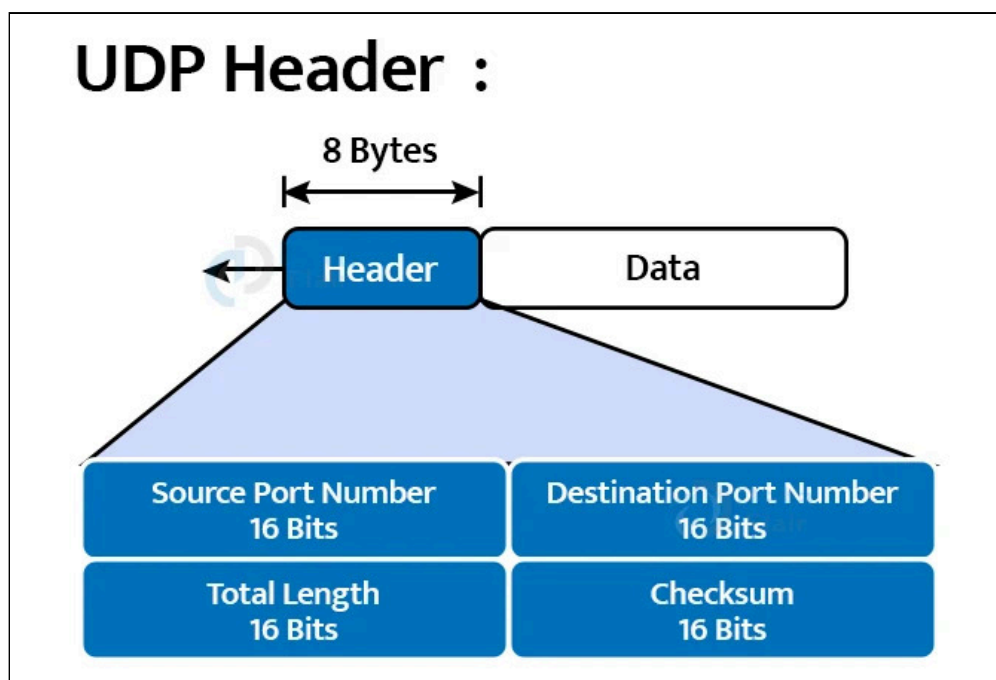


Рисунок 1.1 – Структура заголовка *UDP*-пакета

Система доменных имён по своей природе строится именно на таком взаимодействии. Согласно стандарту [1], основным транспортом для *DNS* является протокол *UDP*, функционирующий на порту 53. *DNS*-пакет имеет собственную структуру, отличную от *UDP*-заголовка, и располагается в полезной нагрузке датаграммы.

Взаимодействие между клиентом и сервером в системе доменных имён строится по классической модели «запрос – ответ» без установления постоянного соединения. Клиент формирует запрос, направляет его на стандартный порт 53 сервера и ожидает ответного пакета. Сервер, в свою очередь, принимает входящие датаграммы, обрабатывает каждую независимо и отправляет ответ непосредственно на адрес отправителя. Такая архитектура клиент-сервер идеально соответствует природе *UDP*: каждый пакет самодостаточен и несёт в себе все необходимые адресные данные для маршрутизации ответа. Порт 53 закреплён за службой *DNS* международным стандартом и является общепризнанным идентификатором данного протокола на транспортном уровне.

1.2 Иерархия *DNS*-серверов и процесс разрешения запросов

Система доменных имён представляет собой распределённую иерархическую базу данных, организованную в виде дерева. Каждый узел этого дерева соответствует домену и обслуживается одним или несколькими серверами, которые хранят авторитетную информацию о нём. Понимание структуры этой иерархии необходимо для осознания того, как происходит процесс разрешения имени в *IP*-адрес.

Корнем всей системы доменных имён является точка (.), обозначаемая как корневой домен или *root*-домен. Серверы, обслуживающие корневой домен, называются *root*-серверами. Их количество, состав и географическое распределение тщательно контролируются для обеспечения устойчивости всей системы. На момент написания работы функционирует 13 логических *root*-серверов, обозначаемых латинскими буквами от А до М, хотя физически каждый из них представляет собой распределённый кластер с множеством географически распределённых экземпляров. *Root*-серверы не хранят информации о конкретных веб-сайтах. Их единственная функция состоит в направлении запросов к серверам следующего уровня – серверам доменов верхнего уровня.

Домены верхнего уровня, обозначаемые аббревиатурой *TLD* (*Top-Level Domain*), образуют второй уровень иерархии. Они делятся на две основные категории. Первую составляют общие домены верхнего уровня (*gTLD* – *generic Top-Level Domain*), к которым относятся такие домены, как *COM*, *ORG*, *NET*, *EDU*, *GOV* и многие другие. Эти домены не привязаны к какой-либо конкретной стране и используются преимущественно коммерческими, общественными и государственными организациями. Вторую категорию образуют национальные домены верхнего уровня (*ccTLD* – *country-code Top-Level Domain*), идентифицирующие конкретную страну или территорию: *RU*, *UA*, *US*, *DE*, *JP* и т.д. Каждая из этих категорий управляется соответствующим администратором: для *gTLD* это корпорация *ICANN*, для *ccTLD* – национальные реестры конкретных стран. Серверы *TLD* хранят информацию о доменах второго уровня, которые регистрируются конечными пользователями или организациями.

Авторитативные серверы (*Authoritative DNS Servers*) образуют последний уровень иерархии, с которым непосредственно взаимодействует резолвер при разрешении имени. Авторитативный сервер – это сервер, который хранит оригинальные записи о домене и имеет право отвечать на запросы об этом домене без обращения к другим серверам. [2] Когда организация регистрирует доменное имя, она указывает список авторитативных серверов для этого домена, которые вносятся в родительскую зону. Например, для домена *example.com* авторитативные серверы указываются в зоне *COM*. Типичная конфигурация включает два или более сервера для обеспечения отказоустойчивости.

Процесс разрешения имени может протекать двумя принципиально разными способами. Первый способ – рекурсивное разрешение. В этом случае клиент (или рекурсивный резолвер, которым обычно выступает *DNS*-сервер провайдера) берёт на себя всю работу по обходу иерархии: он сам обращается к *root*-серверу, затем к *TLD*-серверу, затем к авторитативному серверу и возвращает готовый результат клиенту. Второй способ – итеративное разрешение, при котором сервер отвечает клиенту тем, что знает сам, и указывает, к какому серверу следует обратиться дальше. Именно итеративный подход заложен в основу работы рекурсивных резолверов. Они выполняют серию итеративных запросов к различным серверам иерархии, собирая информацию по частям, пока не получают окончательный ответ. Несмотря на многоэтапность, этот процесс занимает миллисекунды благодаря кэшированию на каждом уровне

иерархии: однажды полученный ответ сохраняется на время, определяемое полем *TTL* соответствующей ресурсной записи.

Необходимо отметить, что система доменных имён спроектирована с расчётом на отказоустойчивость и масштабируемость. Каждый уровень иерархии может содержать множество серверов, и выход из строя отдельного сервера не приводит к отказу всей системы. Делегирование полномочий от родительской зоны к дочерней позволяет равномерно распределить нагрузку между серверами и масштабировать систему по мере роста пространства имён.

1.3 Формат файла зоны

Авторитативный *DNS*-сервер отличается от рекурсивного резолвера тем, что не ищет ответ путём обхода иерархии, а хранит его локально. Источником этих данных служит файл зоны – текстовый файл, содержащий набор ресурсных записей для обслуживаемого домена. Понятие файла зоны закреплено стандартом [1] и реализовано во всех распространённых *DNS*-серверах.

Файл зоны открывается директивами *\$ORIGIN* и *\$TTL*, задающими базовый домен и время жизни записей по умолчанию. Следом располагается обязательная запись *SOA* (Start of Authority), описывающая параметры зоны: имя первичного сервера, серийный номер и интервалы обновления. Далее следуют ресурсные записи, каждая из которых содержит имя узла, класс *IN*, тип и значение. Пример файла зоны приведён на рисунке 1.2.

```
$ORIGIN tld.
$TTL 5m

example      IN      SOA      ns1.example.tld. hostmaster.example.tld. (
                                90          ; serial
                                4h          ; refresh
                                15m         ; retry
                                8h          ; expire
                                4m)         ; negative caching TTL
              IN      NS       ns1.example.tld.
              IN      NS       ns2.example.tld.
              MX      10       mail.example.tld.
              IN      A        127.0.0.1

              IN TXT "v=spf1 a mx a:mail.example.tld a:www.example.tld ?all"

$ORIGIN example.tld.

ns1           IN      A        127.0.0.2
ns2           IN      A        127.0.0.3
www           IN      CNAME     example.tld.
mail          IN      AAAA     ::1
mail          IN      A        127.0.0.4
```

Рисунок 1.2 – Пример файла зоны

При запуске сервер разбирает файл построчно, проверяя корректность каждой записи: соответствие формата адресов стандарту, допустимость символов в доменных именах, принадлежность *TTL* допустимому диапазону. Некорректные строки пропускаются с фиксацией ошибки в журнале, что позволяет серверу продолжить загрузку даже при наличии ошибок в отдельных записях.

1.4 Организация кэша в памяти

Кэш в памяти – это промежуточное хранилище данных, обеспечивающее быстрый доступ к часто запрашиваемым результатам без повторного выполнения дорогостоящих операций их получения. В отличие от дисковых хранилищ, оперативная память обеспечивает доступ к данным за наносекунды, что делает её естественным выбором для хранения рабочего набора данных в высоконагруженных системах. Расплатой за скорость является энергозависимость, ведь содержимое кэша не переживает перезапуск процесса и должно быть восстановлено из первоисточника.

В контексте *DNS*-сервера кэш хранит ресурсные записи, полученные ранее в ответ на запросы клиентов. Это позволяет отвечать на повторяющиеся запросы без обращения к файлу зоны или вышестоящим серверам иерархии. Поскольку множество клиентов периодически запрашивают одни и те же имена, даже небольшой кэш способен значительно снизить нагрузку на сервер.

Ключевой особенностью *DNS*-кэша является ограниченный срок хранения записей, определяемый значением *TTL* [3]. Это отражает природу самой системы доменных имён: записи в зоне могут меняться, и хранение устаревших данных приведёт к выдаче некорректных ответов. Поэтому каждая запись в кэше сопровождается меткой времени добавления, а при обращении к ней проверяется, не истёк ли срок её жизни.

Многопоточная архитектура сервера накладывает дополнительное требование в виде потокобезопасного доступа к кэшу. При этом параллельное чтение из нескольких потоков безопасно, тогда как запись требует монопольного доступа. Разграничение этих двух режимов позволяет избежать состояния гонки, не создавая излишних ограничений на параллельность операций чтения.

2 ПЛАТФОРМА ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ

Операционная система предоставляет абстракции, через которые прикладная программа взаимодействует с аппаратурой, а глубокое понимание этих абстракций является необходимым условием для написания корректного сетевого кода. В данном разделе рассматриваются теоретические основы и обоснование выбора инструментов, применённых при разработке *DNS*-сервера.

2.1 Язык программирования

Среди языков программирования общего назначения *C++* занимает особое место в области системного и сетевого программирования. Главное достоинство языка – прямой контроль над памятью: программист самостоятельно определяет время жизни объектов, управляет размещением данных в стеке или куче и может работать с сырыми байтовыми буферами без дополнительных уровней абстракции. Это свойство принципиально важно при разработке сетевых серверов, где необходимо напрямую формировать и разбирать двоичные пакеты строго определённого формата.

Производительность *C++* обусловлена компиляцией в нативный машинный код без промежуточного исполнения. В сочетании с отсутствием сборщика мусора это обеспечивает предсказуемые задержки – качество, критичное для серверного программного обеспечения, где скачки времени отклика недопустимы. Кроме того, *C++* не вводит дополнительных уровней абстракции между прикладным кодом и ядром операционной системы, поэтому вызовы функций стандартной библиотеки для работы с сокетами транслируются напрямую в соответствующие системные вызовы без промежуточных рантаймов.

Для данного проекта используется стандарт *C++20*, расширивший язык рядом инструментов, существенно повышающих безопасность и выразительность кода. Тип *std::span* представляет невладеющий вид на непрерывную последовательность элементов: он передаёт одновременно указатель и длину, исключая тем самым возможность выхода за границы буфера. Контейнер *std::vector* обеспечивает автоматическое управление временем жизни динамических массивов, а *std::unordered_map* – хранение записей зоны с амортизированным доступом за константное время. Для организации конкурентного доступа к разделяемым данным применяется *std::shared_mutex*, позволяющий различать операции чтения и записи и не блокировать читателей друг относительно друга. Потоки выполнения создаются средствами *std::thread*, входящего в стандартную библиотеку и не требующего внешних зависимостей.

2.2 Операционная система

Операционная система *Linux* является доминирующей платформой для серверного программного обеспечения. Её ядро реализует полный стек сетевых протоколов и предоставляет прикладным программам устойчивый интер-

фейс системных вызовов, основанный на стандарте *POSIX*. Благодаря этому интерфейсу операции над файлами, процессами, сигналами и сетевыми соединениями унифицированы таким образом, что код, написанный для одной *POSIX*-совместимой системы, может быть перенесён на другую с минимальными изменениями.

Для работы с сетью *POSIX* определяет интерфейс сокетов, в который входят системные вызовы *socket*, *bind*, *recvfrom* и *sendto*, доступные на любой *Linux*-системе. Ядро *Linux* реализует эти вызовы с высокой эффективностью, используя внутренние механизмы буферизации, оптимизированные очереди пакетов и встроенную поддержку многоядерной обработки.

Для разработки *DNS*-сервера *Linux* удобен по нескольким причинам. Во-первых, порт 53, стандартный для *DNS*, в пространстве адресов ниже 1024 является привилегированным, а *Linux* предоставляет гибкие механизмы управления привилегиями, позволяющие запускать сервер с минимально необходимым набором прав. Во-вторых, богатая экосистема инструментов командной строки – *dig*, *tcpdump*, *ss* – даёт возможность немедленно проверять поведение сервера на уровне сетевых пакетов. В-третьих, именно на *Linux* сосредоточена основная часть промышленных реализаций *DNS*, что делает эту платформу естественной средой для разработки и тестирования совместимости.

2.3 Принципы сетевого программирования на базе сокетов

Взаимодействие между узлами сети в современных системах строится на основе многоуровневой модели протоколов, где каждый уровень решает строго определённую задачу и предоставляет вышестоящему уровню абстракцию, скрывающую детали реализации. Для прикладного программиста ключевым является интерфейс на границе между транспортным и прикладным уровнями – именно здесь располагается программный интерфейс сокетов.

Сокет представляет собой программную абстракцию конечной точки сетевого обмена: через него приложение отправляет данные в сеть и принимает данные из неё, не зная ничего о маршрутизации, фрагментации или физической среде передачи. Интерфейс *BSD Socket API*, предложенный в рамках реализации *BSD Unix* и вошедший в стандарт *POSIX*, является стандартом де-факто для всех современных операционных систем. Единообразие этого интерфейса означает, что код сетевого приложения остаётся переносимым вне зависимости от конкретной платформы.

Различие между *UDP*- и *TCP*-сокетами носит концептуальный характер. *TCP* предоставляет надёжный поток байт с гарантией доставки, упорядоченностью и контролем перегрузки – это достигается установлением соединения и поддержанием его состояния на обоих концах. *UDP*, напротив, является протоколом без установления соединения: каждая датаграмма независима, отправляется и принимается как единое целое, а гарантии доставки отсутствуют. Для *DNS* такой режим работы является предпочтительным, поскольку запрос и ответ умещаются в один пакет, а повторная попытка при потере пакета реализуется на стороне клиента.

Важным аспектом сетевого программирования является порядок байт. Процессоры с архитектурой *x86* используют порядок «от младшего к старшему» (*little-endian*), тогда как сетевые протоколы требуют порядка «от старшего к младшему» (*big-endian*, или сетевого порядка байт). При формировании числовых полей заголовков *DNS* каждое многобайтовое значение должно быть явно преобразовано с помощью соответствующих стандартных функций, иначе пакет будет некорректно интерпретирован любым другим участником сетевого обмена.

2.4 Работа с бинарными структурами данных и сериализация

Сериализация – это процесс преобразования объектов программы в последовательность байт для хранения или передачи по сети, а десериализация – обратный процесс восстановления объектов из такой последовательности. В контексте сетевых протоколов сериализация является одной из центральных задач: формат передаваемых данных строго регламентирован стандартом, и любое отступление от него делает пакет нечитаемым для других реализаций.

Бинарный формат *DNS*-пакета обладает рядом особенностей, существенно усложняющих сериализацию. Пакет состоит из фиксированного заголовка и нескольких секций переменной длины, поэтому прямое отображение буфера на единую структуру данных невозможно. Доменные имена кодируются в виде последовательности меток, где каждый сегмент предваряется однобайтовым счётчиком его длины, а вся последовательность завершается нулевым байтом. Такое представление требует итеративного разбора и не имеет фиксированного размера.

Дополнительную сложность вносит механизм сжатия имён, предусмотренный стандартом *DNS* для уменьшения размера пакета. Если имя или его часть уже встречалось ранее в том же пакете, вместо повторной записи используется двухбайтовый указатель на смещение внутри пакета. Старшие два бита первого байта отличают указатель от обычного счётчика длины. Логика десериализации обязана распознавать такие указатели и корректно восстанавливать полное имя по цепочке ссылок. При этом необходимо учитывать возможность намеренно сформированных циклических указателей, что требует явного ограничения глубины обхода.

Отдельного внимания требует вопрос выравнивания структур в памяти. Компилятор вправе вставлять незначащие байты между полями структуры для выравнивания по границам машинного слова, что ускоряет доступ к памяти. Однако при работе с сетевыми пакетами это недопустимо: стандарт определяет положение каждого поля с точностью до одного бита, и любое «паразитное» выравнивание нарушает совместимость. Управление выравниванием осуществляется директивами компилятора.

Стандарт *C++20* предоставляет удобные инструменты для безопасной работы с двоичными буферами. Тип `std::span<const uint8_t>` позволяет передавать в функции разбора вид на буфер – одновременно указатель и длину – без копирования и без риска обращения за его пределы. Контейнер

`std::vector<uint8_t>` берёт на себя управление временем жизни буферов ответа, исключая ручное освобождение памяти. Совместное применение этих инструментов позволяет строить надёжный код сериализации, устойчивый к некорректным входным данным.

3 ТЕОРЕТИЧЕСКОЕ ОБОСНОВАНИЕ РАЗРАБОТКИ ПРОГРАММНОГО ПРОДУКТА

Реализация *DNS*-сервера требует детального понимания структуры протокола на бинарном уровне. Каждый байт пакета имеет строго определённое назначение, а отклонение от спецификации делает пакет нечитаемым для других реализаций. В этом разделе рассматриваются теоретические основы формата *DNS*-пакета, кодирования доменных имён, типов ресурсных записей и принципов кэширования.

3.1 Структура DNS-пакета

Протокол *DNS* определён стандартом [1], опубликованным в 1987 году и применяемым до настоящего времени. Пакет состоит из заголовка фиксированного размера и последующих секций переменной длины, как показано на рисунке 3.1. Заголовок содержит метаданные о типе сообщения, тогда как секции данных содержат сам запрос и ответы.



Рисунок 3.1 – Структура DNS-пакета

3.1.1 Заголовок DNS-пакета.

Заголовок *DNS*-пакета имеет фиксированную длину 12 байт и состоит из шести двухбайтовых полей. Каждое поле представлено в сетевом порядке байт (*big-endian*), независимо от архитектуры процессора. Структура заголовка приведена в таблице 3.1.

Таблица 3.1 – Структура заголовка *DNS*-пакета

Поле	Размер, бит	Смещение, байт	Назначение
<i>ID</i>	16	0	Идентификатор транзакции для сопоставления запроса и ответа
<i>QR</i>	1	2	Флаг типа сообщения: 0 – запрос, 1 – ответ

Продолжение таблицы 3.1

Поле	Размер, бит	Смещение, байт	Назначение
<i>Opcode</i>	4	2	Тип операции: 0 – стандартный запрос
<i>AA</i>	1	2	Флаг авторитативного ответа
<i>TC</i>	1	2	Флаг усечения сообщения
<i>RD</i>	1	2	Флаг запроса рекурсии
<i>RA</i>	1	3	Флаг доступности рекурсии на сервере
<i>Z</i>	3	3	Зарезервированные биты, должны быть нулевыми
<i>RCODE</i>	4	3	Код результата: 0 – успех, 3 – <i>NXDOMAIN</i>
<i>QDCOUNT</i>	16	4	Количество записей в секции запроса
<i>ANCOUNT</i>	16	6	Количество записей в секции ответа
<i>NSCOUNT</i>	16	8	Количество записей авторитативных серверов
<i>ARCOUNT</i>	16	10	Количество дополнительных записей

Поле *ID* представляет собой 16-битное беззнаковое целое, генерируемое клиентом случайным образом и копируемое в ответ без изменений. Это позволяет клиенту сопоставить каждый ответ с соответствующим запросом в условиях параллельной обработки множества запросов. При этом два разных клиента могут одновременно отправить запросы с одинаковыми идентификаторами, поэтому сервер различает запросы по паре «адрес клиента, идентификатор».

Флаги управления занимают часть второго и третьего байтов. Поле *QR* (Query/Response) различает запросы и ответы: значение 0 означает запрос, 1 – ответ. Поле *Opcode* определяет тип операции: стандартный запрос кодируется нулём, что является единственным типом, поддерживаемым простыми реализациями. Флаг *AA* (Authoritative Answer) устанавливается сервером в значение 1, если сервер хранит оригинальные данные зоны, и в 0, если ответ получен из кэша или от другого сервера. Флаг *RD* (Recursion Desired) сигнализирует серверу, что клиент ожидает рекурсивного разрешения запроса с обходом иерархии. Флаг *RA* (Recursion Available) устанавливается сервером, поддерживающим рекурсию.

Поле *RCODE* кодирует результат обработки запроса. Значение 0 означает успешное выполнение. Значение 3 (*NXDOMAIN*, Non-Existent Domain) сигнализирует, что запрашиваемое доменное имя не существует в зоне. Значение 2 (Server Failure) указывает на внутреннюю ошибку сервера. Значение 4 (Not Implemented) означает, что сервер не поддерживает запрошенный тип операции.

Счётчики секций (*QDCOUNT*, *ANCOUNT*, *NSCOUNT*, *ARCOUNT*) задают количество записей в соответствующих секциях данных. В типичном запросе *QDCOUNT* равно 1, а остальные счётчики – нулю. В ответе *QDCOUNT*

также равно 1 (запрос дублируется в ответе), *ANCOUNT* содержит количество записей ответа, а поля *NSCOUNT* и *ARCOUNT* используются при делегировании или передаче дополнительной информации.

3.1.2 Секция запроса.

Секция запроса (Question Section) описывает доменное имя, о котором клиент запрашивает информацию. Каждая запись в этой секции состоит из трёх компонентов: кодированного доменного имени (*QNAME*), типа запроса (*QTYPE*) и класса запроса (*QCLASS*).

Доменное имя кодируется в формате меток переменной длины. Каждая метка представляет собой сегмент доменного имени между точками. Например, имя *example.com* состоит из двух меток: «example» и «com». Кодирование начинается с однобайтового счётчика длины метки, за которым следуют символы метки в кодировке *ASCII*. Весь домен завершается нулевым байтом. Имя *example.com* кодируется как: 7, „е“, „х“, „а“, „м“, „п“, „л“, „е“, 3, „с“, „о“, „м“, 0. Длина метки ограничена 63 байтами, что следует из старших двух битов счётчика, зарезервированных для механизма сжатия.

После доменного имени располагается двухбайтовое поле *QTYPE*, определяющее тип запрашиваемой записи. Значения типов приведены в таблице 3.2.

Таблица 3.2 – Типы *DNS*-запросов

Код	Тип	Назначение
1	<i>A</i>	Адрес <i>IPv4</i>
28	<i>AAAA</i>	Адрес <i>IPv6</i>
5	<i>CNAME</i>	Каноническое имя
255	<i>ANY</i>	Все доступные записи

Поле *QCLASS* определяет класс сети. В современных реализациях используется исключительно класс *IN* (Internet), кодируемый значением 1. Другие классы (*CS* для сети *CSNET*, *CH* для сети *Chaos*) являются историческими и не применяются.

3.1.3 Секция ответа.

Секция ответа (Answer Section) содержит одну или несколько ресурсных записей, удовлетворяющих запросу. Каждая запись имеет единый формат, независимо от типа данных. Структура ресурсной записи включает пять компонентов: доменное имя (*NAME*), тип (*TYPE*), класс (*CLASS*), время жизни (*TTL*) и данные (*RDATA*).

Доменное имя кодируется так же, как в секции запроса, но с возможностью применения механизма сжатия. Если имя или его часть уже встречалось в пакете, вместо повторной записи используется двухбайтовый указатель на смещение в пакете. Указатель распознаётся по старшим двум битам, установленным в единицу: если старшие биты байта равны 11, следующие 14 бит

интерпретируются как смещение от начала пакета [4]. Такое сжатие существенно уменьшает размер ответов, содержащих множество записей с одинаковыми суффиксами доменных имён.

Поле *TTL* представляет собой 32-битное беззнаковое целое, определяющее время жизни записи в секундах. Значение устанавливается администратором зоны и указывает, в течение какого времени запись может храниться в кэше без повторного запроса к авторитативному серверу. Для динамически изменяющихся записей *TTL* устанавливается в несколько минут, для статических – в несколько часов или суток.

Поле *RDLLENGTH* содержит длину данных в байтах, что позволяет парсеру пропустить неизвестные типы записей без нарушения разбора остального пакета. Поле *RDATA* содержит сами данные, формат которых зависит от типа записи.

3.2 Типы ресурсных записей

Каждая ресурсная запись в системе доменных имён несёт конкретный тип информации о домене. Реализация простого *DNS*-сервера требует поддержки трёх базовых типов: *A*, *AAAA* и *CNAME*.

3.2.1 Запись типа A.

Запись типа *A* (Address) связывает доменное имя с *IPv4*-адресом узла. Поле *RDATA* такой записи содержит ровно 4 байта – представление адреса в бинарном виде. Например, адрес 192.0.2.1 кодируется как последовательность байт 0xC0, 0x00, 0x02, 0x01. Порядок байт – сетевой, то есть от старшего к младшему. Одному доменному имени может соответствовать несколько *A*-записей, что обеспечивает распределение нагрузки между серверами.

3.2.2 Запись типа AAAA.

Запись типа *AAAA* (Address, version 6) связывает доменное имя с *IPv6*-адресом. Поле *RDATA* содержит 16 байт – полное представление *IPv6*-адреса. Например, адрес 2001:db8::1 кодируется как 20 01 0d b8 00 00 00 00 00 00 00 00 00 00 00 01. Применение сжатия нулей, допустимое в текстовой форме *IPv6*-адресов, не используется в бинарном представлении: все 16 байт передаются явно.

С распространением протокола *IPv6* большинство доменов имеют одновременно *A*- и *AAAA*-записи, что обеспечивает доступность сервиса как для клиентов с *IPv4*, так и для клиентов с *IPv6*. Клиент, поддерживающий оба протокола, обычно запрашивает оба типа записей параллельно и использует тот адрес, который быстрее отвечает.

3.2.3 Запись типа CNAME.

Запись типа *CNAME* (Canonical Name) определяет псевдоним доменного имени. В отличие от *A*- и *AAAA*-записей, содержащих конечные данные (адреса), *CNAME* содержит ссылку на другое доменное имя. Поле *RDATA* содержит

доменное имя в том же формате меток, что и в секции запроса, с применением сжатия при необходимости.

Наличие *CNAME*-записи означает, что для получения адреса клиент должен выполнить второй запрос, на этот раз к каноническому имени. Однако хорошо спроектированный сервер возвращает в одном ответе как *CNAME*-запись, так и соответствующую *A*- или *AAAA*-запись, что исключает необходимость повторного запроса.

Важным ограничением является запрет на наличие других записей для имени, имеющего *CNAME*. Если для домена определён псевдоним, то *A*-, *AAAA*-, *MX*- и другие записи для этого имени не допускаются. Каноническое имя, на которое указывает *CNAME*, может иметь любые записи.

3.3 Принципы кэширования DNS-записей

Кэширование является ключевым механизмом повышения производительности системы доменных имён. Без кэширования каждый запрос к популярному домену требовал бы прохождения через всю иерархию серверов, что приводило бы к неприемлемым задержкам и перегрузке корневых серверов. Время жизни записи и механизм её инвалидации строго регламентированы стандартом.

3.3.1 Время жизни записи и его назначение.

Поле *TTL* (*Time To Live*) в каждой ресурсной записи определяет максимальное время в секундах, в течение которого запись может храниться в кэше. Значение *TTL* устанавливается администратором авторитативной зоны и отражает компромисс между снижением нагрузки на сервер и актуальностью данных.

Для статических записей, таких как адреса веб-серверов крупных сайтов, *TTL* устанавливается в диапазоне от нескольких часов до одних суток. Это означает, что изменение адреса сервера будет видно всем клиентам не мгновенно, а лишь после истечения установленного времени. Для динамических записей, изменяющихся в процессе балансировки нагрузки или в ходе обслуживания, *TTL* может составлять несколько минут или даже секунд.

При получении записи с авторитативного сервера кэш фиксирует момент её сохранения. Каждый последующий запрос к этой записи сопровождается проверкой: если разность между текущим временем и временем сохранения превышает *TTL*, запись считается устаревшей и удаляется из кэша. Следующий запрос к этому же домену потребует повторного обращения к авторитативному серверу.

3.3.2 Позитивное и негативное кэширование.

Кэширование в системе доменных имён делится на два вида: позитивное и негативное. Позитивное кэширование применяется к существующим записям — тем, для которых сервер вернул успешный ответ с кодом результата 0. Такая запись сохраняется в кэше вместе с её *TTL*, и последующие запросы к

тому же имени и типу обрабатываются напрямую из кэша без обращения к серверу.

Негативное кэширование применяется к несуществующим доменам. Когда авторитативный сервер отвечает кодом *NXDOMAIN*, это означает, что запрашиваемое имя не существует в зоне. Без кэширования такого результата каждый запрос к несуществующему домену проходил бы через всю цепочку серверов, что создавало бы значительную нагрузку при атаках, использующих случайные поддомены. Стандарт [5] вводит механизм кэширования отрицательных ответов. Время жизни негативного ответа определяется полем *TTL* записи *SOA* в авторитативной зоне или явным значением, переданным в ответе.

Негативное кэширование позволяет значительно снизить количество запросов к авторитативным серверам в случае ошибок в настройках клиентов или при сканировании несуществующих доменов. В реализации простого сервера негативное кэширование требует хранения не только самих записей, но и информации о том, что для данного имени и типа был получен отрицательный ответ.

3.3.3 Управление размером кэша.

Неограниченное накопление записей в кэше приводит к исчерпанию оперативной памяти сервера. Для предотвращения этого применяются стратегии управления размером кэша. Простейшая стратегия – установка максимального количества записей. При достижении лимита старые записи вытесняются новыми по принципу *LRU* (Least Recently Used): удаляется запись, к которой дольше всего не обращались. Реализация данного алгоритма на основе связки хэш-таблицы и двусвязного списка обеспечивает константную сложность операций вставки и поиска [6].

Более совершенные реализации используют адаптивные алгоритмы, учитывающие частоту обращений к записи и оставшееся время жизни. Запись с высоким *TTL* и редкими обращениями является кандидатом на вытеснение в первую очередь, тогда как запись с малым *TTL* и частыми обращениями сохраняется до истечения времени жизни.

Периодическая очистка истекших записей необходима для освобождения памяти. Наивная реализация проверяет все записи кэша при каждом обращении, что приводит к линейной сложности операции. Эффективная реализация использует отдельный поток, периодически обходящий кэш и удаляющий истекшие записи, либо ленивую очистку, при которой проверка выполняется только при обращении к конкретной записи.

4 ПРОЕКТИРОВАНИЕ ФУНКЦИОНАЛЬНЫХ ВОЗМОЖНОСТЕЙ ПРОГРАММЫ

Реализация *DNS*-сервера требует разработки чёткого алгоритма обработки входящих запросов. В предыдущих разделах были рассмотрены архитектурные решения, структура протокола и организация хранения данных. Настоящий раздел посвящён проектированию логики обработки запроса: от момента получения пакета до формирования и отправки ответа клиенту.

4.1 Алгоритм разбора входящего *DNS*-запроса

Первым этапом обработки запроса является извлечение структурированной информации из бинарного потока данных, полученного по протоколу *UDP*. Пакет представляет собой последовательность байт, в которой каждый элемент имеет строго определённое положение и назначение в соответствии со стандартом [1].

Процесс разбора начинается с извлечения заголовка *DNS*-пакета. Первые 12 байт полученной датаграммы интерпретируются как структура фиксированного формата. Поле *ID* извлекается из байтов 0–1 с учётом сетевого порядка следования байт. Данный идентификатор должен быть скопирован в ответ без изменений, что позволяет клиенту сопоставить ответ с исходным запросом. Байты 2 и 3 содержат набор флагов: бит *QR* определяет тип сообщения, поле *Opcode* – тип операции, флаг *RD* – требование рекурсии. Проверка флага *QR* позволяет отфильтровать некорректные пакеты: если значение не равно нулю, пакет является ответом, а не запросом, и должен быть отброшен. Поле *RCODE* в запросе должно быть нулевым, иное значение сигнализирует о нарушении протокола.

Счётчики секций (*QDCOUNT*, *ANCOUNT*, *NSCOUNT*, *ARCOUNT*) извлекаются из байтов 4–11. Корректный стандартный запрос содержит ровно одну запись в секции вопросов, поэтому значение *QDCOUNT* должно равняться единице. Счётчики секций ответа в запросе должны быть нулевыми. Если эти условия не выполняются, пакет считается некорректным и отбрасывается с фиксацией ошибки в журнале.

После извлечения и валидации заголовка следует разбор секции запроса. Эта секция начинается с 13-го байта пакета и содержит кодированное доменное имя (*QNAME*), за которым следуют двухбайтовые поля *QTYPE* и *QCLASS*. Декодирование доменного имени осуществляется путём последовательного чтения меток. Каждая метка начинается с байта длины, определяющего количество следующих за ним символов. Значение длины от 1 до 63 указывает на обычную метку, значение 0 означает конец имени. Если старшие два бита байта длины установлены в единицу, следующие 14 бит интерпретируются как смещение для механизма сжатия. Извлечённые метки объединяются в полное доменное имя путём конкатенации через символ точки.

Поле *QTYPE*, расположенное сразу после кодированного имени, определяет тип запрашиваемой ресурсной записи. Значения 1, 28 и 5 соответствуют

типам *A*, *AAAA* и *CNAME*. Значение 255 означает запрос всех доступных записей. Поле *QCLASS* должно иметь значение 1, соответствующее классу *IN*. Иные значения, как правило, не поддерживаются и приводят к формированию ответа с кодом ошибки.

Результатом процесса разбора является структура данных, содержащая идентификатор запроса, извлечённое доменное имя, тип и класс запроса. Эта информация передаётся на следующий этап обработки – поиск соответствующих записей.

4.2 Двухуровневая система поиска ресурсных записей

Обработка запроса после этапа разбора переходит к поиску соответствующих ресурсных записей. Поиск осуществляется в два этапа: сначала в кэше оперативной памяти, затем – при отсутствии данных в кэше – в загруженном файле зоны. Такая двухуровневая архитектура обеспечивает баланс между скоростью доступа и полнотой данных.

4.2.1 Поиск в кэше и проверка актуальности записей.

Первым этапом поиска является обращение к кэшу. Ключом поиска служит пара значений: доменное имя и тип записи. Если ранее был выполнен запрос с теми же параметрами, результат мог быть сохранён в кэше, и повторное обращение к файлу зоны не требуется.

Кэш представляет собой ассоциативный контейнер, в котором каждая запись сопровождается меткой времени добавления и значением *TTL*. При обнаружении записи выполняется проверка её актуальности: вычисляется разность между текущим временем и временем сохранения записи. Если эта разность превышает значение *TTL*, запись считается устаревшей и удаляется из кэша. Запрос обрабатывается так, как если бы записи в кэше не было.

Если запись найдена и не истекла, она извлекается из кэша и передаётся на этап формирования ответа. При этом в ответе должно быть указано оставшееся время жизни записи: из исходного значения *TTL* вычитается время, прошедшее с момента сохранения. Это гарантирует, что клиент не будет хранить запись дольше, чем предусмотрено авторитативным сервером.

Успешное обнаружение записи в кэше завершает процесс поиска, и выполнение переходит к этапу формирования ответа. Если запись не найдена или оказалась устаревшей, процесс переходит ко второму этапу – поиску в файле зоны.

4.2.2 Поиск в файле зоны и обработка результата.

Файл зоны содержит полный набор авторитативных записей для обслуживаемого домена. Данные файла зоны загружаются в память при запуске сервера и организованы в виде структуры данных, обеспечивающей эффективный поиск по доменному имени.

Поиск в зоне осуществляется по доменному имени, извлечённому из запроса. Если домен найден, извлекаются все ресурсные записи, ассоцииро-

ванные с этим именем. Далее выполняется фильтрация по типу: если клиент запрашивал тип *A*, отбираются только записи этого типа. Если запрашивался тип *ANY*, возвращаются все записи домена независимо от типа.

Особый случай представляет обработка записей типа *CNAME*. Если для запрашиваемого имени существует запись *CNAME*, сервер должен вернуть её клиенту и, по возможности, дополнить ответ записями канонического имени. Для этого выполняется второй поиск – уже по каноническому имени, на которое указывает *CNAME*. Если каноническое имя находится в той же зоне, соответствующие *A*- или *AAAA*-записи добавляются в секцию ответа. Такое поведение исключает необходимость повторного запроса со стороны клиента.

Найденные записи формируют список результатов, который передаётся на этап построения ответа. Дополнительно эти записи сохраняются в кэше с указанием текущего времени и значения *TTL* из зоны, что позволяет обработать последующие идентичные запросы без обращения к файлу зоны.

Если ни одной записи для запрашиваемого имени в зоне не обнаружено, процесс переходит к обработке несуществующего домена.

4.3 Формирование DNS-ответа

Этап формирования ответа преобразует структурированные данные, полученные на предыдущих этапах, в бинарное представление *DNS*-пакета, готовое к отправке клиенту. Структура ответа соответствует структуре запроса: заголовок фиксированного размера, за которым следуют секции данных.

Построение заголовка ответа начинается с копирования идентификатора из запроса. Поле *ID* переносится без изменений, что позволяет клиенту сопоставить ответ с запросом. Флаг *QR* устанавливается в значение 1, сигнализируя, что пакет является ответом. Поле *Opcode* копируется из запроса. Флаг *AA* (Authoritative Answer) устанавливается в единицу, если запись была получена из файла зоны, и в ноль, если запись извлечена из кэша. Флаг *RD* копируется из запроса. Флаг *RA* (Recursion Available) устанавливается в ноль, поскольку в простой реализации рекурсивное разрешение запросов, как правило, не выполняется. Поле *RCODE* устанавливается в значение 0 при успешном выполнении запроса.

Счётчики секций заполняются в соответствии с количеством записей в ответе. Поле *QDCOUNT* устанавливается в значение 1, поскольку запрос дублируется в ответе. Поле *ANCOUNT* содержит количество найденных ресурсных записей. Поля *NSCOUNT* и *ARCOUNT*, как правило, устанавливаются в ноль в упрощённых реализациях.

После заголовка в пакет записывается секция запроса. Она содержит те же данные, что были получены от клиента: кодированное доменное имя, тип и класс запроса. Эта секция позволяет клиенту однозначно идентифицировать, на какой вопрос получен ответ, особенно в случае параллельной обработки множества запросов.

Секция ответа формируется путём последовательной сериализации каждой найденной ресурсной записи. Запись начинается с кодирования доменного

имени. Для уменьшения размера пакета применяется механизм сжатия: если доменное имя совпадает с именем в секции запроса, вместо повторного кодирования используется двухбайтовый указатель на смещение 12 (начало секции запроса). Старшие два бита указателя устанавливаются в единицу, остальные 14 бит содержат смещение.

За именем следуют двухбайтовые поля *TYPE* и *CLASS*, определяющие тип и класс записи. Поле *TTL* записывается как 32-битное беззнаковое целое в сетевом порядке байт. Для записей из кэша значение *TTL* уменьшается на время, прошедшее с момента сохранения. Поле *RDLLENGTH* содержит длину данных, следующих за ним. Для записи типа *A* это значение равно 4, для типа *AAAA* – 16. Поле *RDATA* содержит сами данные: адрес *IPv4*, адрес *IPv6* или кодированное доменное имя для записи *CNAME*.

Сформированный пакет отправляется клиенту по протоколу *UDP* на адрес и порт, указанные в качестве источника входящего запроса. Использование *UDP* позволяет серверу не хранить состояние соединения: каждый запрос обрабатывается независимо, а ответ отправляется непосредственно на адрес отправителя.

4.4 Обработка несуществующих доменов и негативное кэширование

Запрос к несуществующему домену является частым случаем в реальной эксплуатации. Ошибки в конфигурации клиентов, опечатки в адресах, атаки на основе случайных поддоменов – все эти ситуации приводят к запросам имён, отсутствующих в зоне. Корректная обработка таких запросов требует формирования ответа специального вида и, при необходимости, сохранения отрицательного результата в кэше.

4.4.1 Формирование ответа *NXDOMAIN*.

Когда поиск в файле зоны не обнаружил ни одной записи для запрашиваемого доменного имени, сервер формирует ответ с кодом результата *NXDOMAIN*. Согласно стандарту [1], этот код имеет числовое значение 3 и означает, что запрашиваемое доменное имя не существует.

Формирование ответа *NXDOMAIN* отличается от формирования успешного ответа установкой поля *RCODE* в заголовке. Все остальные компоненты заголовка заполняются стандартным образом: идентификатор копируется из запроса, флаг *QR* устанавливается в единицу, флаг *AA* устанавливается в единицу, так как сервер авторитативен для зоны и может достоверно утверждать, что домен не существует. Счётчик *QDCOUNT* устанавливается в значение 1, счётчик *ANCOUNT* – в ноль, поскольку секция ответа остаётся пустой.

Секция запроса дублируется в ответе так же, как и в случае успешного ответа. Секция ответа остаётся пустой, что сигнализирует клиенту об отсутствии данных. Клиент, получив пакет с кодом *NXDOMAIN*, интерпретирует это как окончательное отсутствие запрашиваемого домена в зоне.

Следует отличать код *NXDOMAIN* от другого типа отрицательного ответа – *NODATA*. Если доменное имя существует в зоне, но для него нет записей запрашиваемого типа, код результата остаётся нулевым, однако секция ответа остаётся пустой. Например, если клиент запрашивает *AAAA*-запись для домена, у которого есть только *A*-записи, сервер возвращает успешный ответ с пустой секцией ответа. Код *NXDOMAIN* используется исключительно в случае отсутствия самого доменного имени.

4.4.2 Негативное кэширование.

Стандарт [5] вводит понятие негативного кэширования – сохранения информации о несуществующих доменах или отсутствующих типах записей. Без такого механизма каждый повторный запрос к несуществующему домену проходил бы через весь цикл обработки, включая поиск в файле зоны, что создаёт излишнюю нагрузку на сервер.

Негативное кэширование осуществляется путём сохранения в кэше специальной записи, указывающей на отсутствие данных для конкретной пары «доменное имя, тип запроса». Время жизни негативной записи определяется значением *TTL* из записи *SOA* авторитативной зоны или устанавливается в фиксированное значение по умолчанию, например, 300 секунд.

При получении повторного запроса к тому же несуществующему домену сервер обнаруживает негативную запись в кэше и формирует ответ *NXDOMAIN* без обращения к файлу зоны. Это значительно снижает время обработки запроса и уменьшает нагрузку на систему хранения данных.

Негативное кэширование требует осторожности при реализации. Если домен был создан в зоне после сохранения негативной записи, клиенты будут получать устаревший отрицательный ответ до истечения *TTL*. Поэтому время жизни негативных записей обычно устанавливается меньшим, чем для позитивных. Типичное значение составляет от нескольких минут до получаса, что обеспечивает баланс между производительностью и актуальностью данных.

При изменении содержимого зоны, например, при добавлении новых доменов, выполняется очистка кэша или перезапуск сервера. Это гарантирует, что новые записи станут доступны клиентам немедленно, а не после истечения времени жизни негативных записей.

4.5 Многопоточная архитектура сервера

Обеспечение высокой пропускной способности *DNS*-сервера требует параллельной обработки множественных входящих запросов. Последовательная обработка, при которой сервер принимает новый запрос только после завершения обработки предыдущего, приводит к увеличению времени ожидания клиентов и снижению общей производительности системы. Многопоточная архитектура позволяет эффективно использовать вычислительные ресурсы многоядерных процессоров и обеспечивает одновременное обслуживание нескольких запросов.

4.5.1 Модель «приёмник – пул потоков».

Реализация многопоточности основывается на архитектурном паттерне «приёмник – пул потоков» [7]. В данной модели существует главный поток, отвечающий за приём входящих *UDP*-датаграмм, и фиксированное количество рабочих потоков, осуществляющих обработку запросов. Разделение ответственности между приёмом и обработкой обеспечивает высокую отзывчивость сервера: главный поток не блокируется на время выполнения операций разбора, поиска и формирования ответа, а немедленно возвращается к ожиданию следующего запроса.

Главный поток выполняет бесконечный цикл приёма данных из сокета. Получив датаграмму, он помещает пакет вместе с адресом отправителя в потокобезопасную очередь задач и продолжает ожидание следующего запроса. Рабочие потоки, в свою очередь, извлекают задачи из очереди, выполняют полный цикл обработки запроса и отправляют ответ клиенту. Если очередь пуста, рабочие потоки блокируются до появления новой задачи, что исключает излишнее потребление процессорного времени в режиме ожидания.

Количество рабочих потоков определяется на этапе инициализации сервера и передаётся в качестве параметра конфигурации при запуске приложения. Это позволяет пользователю гибко настраивать производительность сервера в зависимости от доступных вычислительных ресурсов и ожидаемой нагрузки. Если параметр не задан явно, используется значение по умолчанию, обеспечивающее стабильную работу на большинстве современных систем. Создание избыточного количества потоков приводит к непроизводительным затратам на переключение контекста, тогда как недостаточное количество не позволяет полностью использовать вычислительные ресурсы.

4.5.2 Механизм корректного завершения потоков.

Управление жизненным циклом рабочих потоков осуществляется с помощью стандартного класса *std::thread*. При инициализации пула потоки запускаются и выполняют цикл ожидания задач. Для обеспечения корректного завершения работы сервера используется механизм сигнализации через атомарный флаг остановки.

Флаг остановки представляет собой логическую переменную, доступ к которой синхронизирован для предотвращения состояний гонки. Когда главный поток инициирует завершение работы, он устанавливает значение флага в состояние «истина» под блокировкой мьютекса и оповещает все ожидающие потоки через условную переменную. Рабочие потоки, пробудившись, проверяют состояние флага и наличие оставшихся задач в очереди. Если пул остановлен и очередь пуста, поток завершает выполнение функции и выходит из цикла.

Главный поток в деструкторе пула потоков ожидает завершения каждого рабочего потока с помощью метода *join()*. Это гарантирует, что все ресурсы будут освобождены, а задачи, находящиеся в процессе выполнения, будут доведены до конца перед полным закрытием приложения. Такой подход обеспечивает надёжность и предотвращает аварийное завершение программы [8].

4.5.3 Потокобезопасная очередь задач.

Взаимодействие между главным потоком и рабочими потоками осуществляется через потокобезопасную очередь задач. Очередь представляет собой обёртку над стандартным контейнером *std::queue*, дополненную механизмами синхронизации.

Защита внутреннего состояния очереди обеспечивается мьютексом *std::mutex*. Каждая операция вставки или извлечения элемента выполняется под блокировкой мьютекса, что исключает возможность гонки данных. Использование блокировки с автоматическим освобождением (*std::unique_lock*) гарантирует освобождение мьютекса даже в случае возникновения исключения.

Механизм ожидания появления элементов в очереди реализуется с помощью условной переменной *std::condition_variable*. Когда рабочий поток обнаруживает, что очередь пуста, он вызывает метод *wait()* условной переменной, передавая ей захваченный мьютекс и предикат. Предикат проверяет условие выхода: наличие задачи в очереди или установку флага остановки. Это атомарно освобождает мьютекс и переводит поток в режим ожидания. Главный поток, добавив элемент в очередь, вызывает метод *notify_one()* или *notify_all()*, что пробуждает ожидающие рабочие потоки. Пробуждённый поток автоматически повторно захватывает мьютекс и продолжает выполнение, если условие предиката истинно. Такой механизм исключает проблему «ложных пробуждений» и обеспечивает эффективное использование ресурсов процессора.

4.5.4 Синхронизация доступа к кэшу.

Кэш *DNS*-записей представляет собой разделяемый ресурс, к которому обращаются все рабочие потоки. Без должной синхронизации одновременный доступ к структуре данных кэша приводит к неопределённому поведению программы, включая повреждение данных и аварийное завершение.

Для синхронизации доступа к кэшу используется механизм *reader-writer lock*, реализованный с помощью класса *std::shared_mutex*. Данный тип мьютекса позволяет разграничить операции чтения и записи: множество потоков могут одновременно удерживать блокировку на чтение, но только один поток может удерживать блокировку на запись, при этом все операции чтения блокируются.

Операции поиска записи в кэше выполняются под разделяемой блокировкой *std::shared_lock*. Поскольку поиск не изменяет структуру кэша, несколько рабочих потоков могут одновременно выполнять операции чтения, что обеспечивает высокую степень параллелизма и минимизирует конкуренцию за доступ к кэшу.

Операции вставки новых записей и удаления устаревших записей требуют эксклюзивной блокировки *std::unique_lock*. Такая блокировка гарантирует, что в момент модификации кэша ни один другой поток не осуществляет чтение или запись, что исключает возможность повреждения внутренней структуры контейнера.

Проверка актуальности записи, включающая сравнение времени жизни с текущим временем, выполняется под разделяемой блокировкой. Если запись оказывается устаревшей, блокировка повышается до эксклюзивной для выполнения удаления. Повышение блокировки требует её освобождения и повторного захвата в эксклюзивном режиме, что создаёт окно, в течение которого другой поток может модифицировать кэш. Поэтому после повторного захвата необходимо повторно проверить наличие записи и её актуальность.

4.5.5 Разделение операций приёма и обработки.

Разделение ответственности между приёмом данных и их обработкой является ключевым архитектурным решением, обеспечивающим высокую пропускную способность сервера. Главный поток выполняет только минимально необходимые операции: вызов системной функции *recvfrom()*, копирование полученных данных в буфер и помещение задачи в очередь. Все операции, требующие вычислительных затрат – разбор пакета, поиск в кэше, поиск в зоне, формирование ответа – делегируются рабочим потокам.

Такое разделение позволяет главному потоку возвращаться к ожиданию следующего запроса максимально быстро, что исключает потерю пакетов при высокой интенсивности входящего трафика. Операционная система буферизует входящие *UDP*-датаграммы в очереди сокета, однако размер этой очереди ограничен. Если главный поток задерживается на обработке запроса, очередь переполняется, и новые пакеты отбрасываются на уровне ядра операционной системы. Немедленное извлечение пакета из очереди сокета и передача его в очередь задач уменьшает вероятность переполнения и повышает надёжность обслуживания.

Рабочие потоки, в свою очередь, не взаимодействуют с сокетом на этапе приёма данных. Они получают уже извлечённые датаграммы из очереди задач, что устраняет необходимость синхронизации доступа к сокету. Отправка ответа выполняется каждым рабочим потоком независимо с использованием функции *sendto()*, которая, как правило, является потокобезопасной для операций записи в различные адреса назначения.

4.6 Логирование и диагностика

Диагностика работы сервера и анализ поступающих запросов требуют ведения журнала событий. Логирование позволяет отслеживать корректность обработки запросов, выявлять ошибки в конфигурации зоны, анализировать эффективность кэша и диагностировать сетевые проблемы. Структурированное ведение журнала является неотъемлемой частью промышленной реализации серверного программного обеспечения.

4.6.1 Вывод информации о полученном запросе.

Каждый принятый запрос регистрируется в журнале с указанием параметров запроса и адреса клиента. Запись включает метку времени, *IP*-адрес

клиента, запрашиваемое доменное имя и тип запроса. Формат записи обеспечивает быстрый поиск и фильтрацию событий при анализе журнала.

Адрес клиента извлекается из структуры *sockaddr*, возвращаемой функцией *recvfrom()*. Для адресов *IPv4* используется функция *inet_ntop()*, преобразующая бинарное представление адреса в текстовую форму. Полученная строка включается в запись журнала, что позволяет идентифицировать источник запроса. Такая информация полезна при диагностике некорректных запросов, анализе географического распределения клиентов и выявлении попыток несанкционированного доступа.

Доменное имя и тип запроса извлекаются в процессе разбора *DNS*-пакета. Запись в журнал формируется после успешного извлечения этих данных, но до выполнения поиска в кэше или зоне. Это гарантирует, что даже запросы, приводящие к ошибкам на последующих этапах обработки, будут зафиксированы в журнале.

Типичная запись журнала имеет вид: «Получен запрос от *192.0.2.1*: домен *example.com*, тип *A*». Такая структура обеспечивает удобочитаемость при ручном анализе и позволяет автоматизировать обработку журнала средствами регулярных выражений или специализированных инструментов анализа логов.

4.6.2 Логирование hit/miss кэша.

Эффективность кэша является важным показателем производительности сервера. Фиксация событий попадания и промахов кэша позволяет оценить долю запросов, обслуживаемых без обращения к файлу зоны, и принять решение о необходимости настройки параметров кэширования.

Событие попадания в кэш (*cache hit*) регистрируется, когда запрошенная пара «доменное имя, тип записи» обнаружена в кэше, и соответствующая запись не истекла. Запись в журнале содержит доменное имя и тип запроса, а также указание на успешное извлечение данных из кэша. Типичный формат: «Кэш: попадание для *example.com (A)*».

Событие промаха кэша (*cache miss*) регистрируется, когда запрошенная запись отсутствует в кэше или оказалась устаревшей. Запись в журнале указывает на необходимость обращения к файлу зоны. Формат записи: «Кэш: промах для *example.com (A)*».

Анализ соотношения попаданий и промахов кэша позволяет оценить эффективность распределения памяти, выделенной под кэш. Высокая доля промахов может свидетельствовать о недостаточном размере кэша или о коротком времени жизни записей. Напротив, чрезмерно большой кэш с незначительным количеством промахов может указывать на излишнее потребление памяти без соответствующего выигрыша в производительности.

4.6.3 Логирование ошибок парсинга.

Некорректные или повреждённые пакеты, поступающие на сервер, должны быть зафиксированы в журнале для последующего анализа. Ошибки парсинга могут возникать вследствие нарушения протокола клиентом, повре-

ждения данных при передаче по сети или целенаправленных попыток эксплуатации уязвимостей.

Наиболее частыми причинами ошибок парсинга являются: некорректное значение счётчика секций запроса (*QDCOUNT* не равен единице), установленный флаг *QR* в запросе (пакет является ответом, а не запросом), недостаточная длина пакета (размер датаграммы меньше минимального размера *DNS*-заголовка), некорректное кодирование доменного имени (указатель на несуществующее смещение или циклическая ссылка в механизме сжатия).

При обнаружении ошибки парсинга формируется запись в журнале с указанием типа ошибки, адреса клиента и, по возможности, содержимого некорректного поля. Типичный формат записи: «Ошибка парсинга: некорректный *QDCOUNT* (значение: 2) от 192.0.2.1». Если разбор пакета не удалось завершить из-за нарушения структуры, в журнал записывается общее описание ошибки: «Ошибка парсинга: пакет слишком короткий от 192.0.2.1».

Фиксация ошибок парсинга позволяет выявить некорректно настроенные клиенты, обнаружить сетевые проблемы, вызывающие повреждение данных, и зафиксировать попытки атак на сервер. Частое появление ошибок от одного и того же адреса может служить основанием для блокировки источника на уровне межсетевого экрана.

4.6.4 Формат логов.

Унифицированный формат записей журнала обеспечивает удобство ручного анализа и автоматической обработки. Каждая запись начинается с метки времени, за которой следует уровень сообщения и текст события.

Метка времени формируется с точностью до секунды в формате *ISO 8601*: «ГГГГ-ММ-ДД ЧЧ:ММ:СС». Использование стандартного формата позволяет корректно сортировать записи и интегрировать журнал с системами централизованного сбора логов. Метка времени извлекается с использованием функций стандартной библиотеки *C++*, таких как *std::chrono::system_clock::now()* для получения календарного времени и *std::put_time()* для его форматирования.

Для измерения интервалов времени в механизме кэширования используется монотонный таймер *std::chrono::steady_clock*, что исключает ошибки при изменении системного времени или переходе на летнее/зимнее время. Это гарантирует корректный расчёт оставшегося *TTL* записей независимо от внешних корректировок времени.

Уровень сообщения классифицирует событие по степени важности. Выделяются следующие уровни: *INFO* – информационные сообщения о нормальной работе сервера (получение запроса, попадание в кэш), *WARNING* – предупреждения о нештатных ситуациях, не приводящих к отказу обслуживания (промах кэша, запрос несуществующего домена), *ERROR* – ошибки, препятствующие корректной обработке запроса (ошибки парсинга, недоступность файла зоны).

Типичная структура записи журнала: «2026-03-25 14:32:01 [*INFO*] Получен запрос от 192.0.2.1: домен *example.com*, тип *A*». Уровень сообщения

заключается в квадратные скобки для визуального выделения, что упрощает фильтрацию записей при просмотре журнала.

Вывод журнала осуществляется в стандартный поток вывода или в файл, в зависимости от конфигурации сервера. При записи в файл используются механизмы ротации логов, предотвращающие неограниченный рост размера журнала. Для этого могут применяться внешние инструменты, такие как *logrotate*, или встроенные механизмы библиотеки логирования.

5 АРХИТЕКТУРА РАЗРАБАТЫВАЕМОЙ ПРОГРАММЫ

В предыдущих разделах были рассмотрены теоретические основы работы *DNS*-сервера, архитектурные решения, лежащие в основе проектирования системы, выбор технологической платформы и проектирование функциональных возможностей. Настоящий раздел посвящён детальному описанию архитектуры программы: взаимодействию компонентов, потоку данных и алгоритмическим решениям, обеспечивающим корректное и эффективное обслуживание *DNS*-запросов в многопоточной среде.

5.1 Функциональная схема обработки запросов и кэширования

Функциональная схема алгоритма отражает последовательность основных этапов обработки *DNS*-запросов. Архитектура сервера базируется на модели пула потоков (*Thread Pool*) и включает семь ключевых функциональных блоков, обеспечивающих полный жизненный цикл обработки пакета.

Первый блок отвечает за прием и буферизацию сетевого *DNS*-запроса. На данном этапе сервер прослушивает сетевой интерфейс и считывает входящие данные в промежуточный буфер, подготавливая их для обработки.

Второй блок осуществляет распределение задачи в очередь потоков. Полученный запрос инкапсулируется в объект задачи и помещается в общую очередь, откуда его извлекает свободный поток из пула для параллельного выполнения.

Третий блок выполняет декодирование и синтаксический анализ пакета. Входящий массив байтов преобразуется во внутренние структуры данных. Проверяется соответствие формата спецификации *RFC 1035*, извлекаются заголовки и секция вопроса.

Четвертый блок отвечает за проверку наличия данных в оперативном кэше. Сформированный ключ запроса проверяется в *LRU*-структуре. Если запись обнаружена, выполняется проверка времени ее жизни (*TTL*). В случае актуальности данных они извлекаются для немедленного формирования ответа. Если же время жизни записи истекло, она удаляется из структуры кэша, а управление передается следующему блоку поиска (возврат пустого результата).

Пятый блок выполняет поиск записей в загруженных зонах. Если данные отсутствуют в кэше или запись была удалена как устаревшая, сервер обращается к авторитативному хранилищу. Здесь производится сопоставление доменного имени и типа записи с данными, загруженными из конфигурационных файлов зон.

Шестой блок осуществляет формирование и сериализацию ответного пакета. Найденные данные упаковываются в секции ответа, авторитативных записей и дополнительных данных. Применяется механизм сжатия имен для минимизации размера пакета.

Седьмой блок выполняет отправку сформированного ответа клиенту. Итоговый бинарный пакет передается через сетевой сокет отправителю запроса, после чего рабочий поток освобождается.

5.2 Описание алгоритма работы сервера

Блок-схема алгоритма представлена в приложении и описывает логику функционирования системы в виде непрерывного потока операций и условий.

Процесс обработки инициируется приемом и последующим декодированием входящего *DNS*-запроса. Сразу после десериализации данных выполняется проверка валидности пакета. Если структура запроса нарушена или не соответствует стандартам, алгоритм переходит к формированию ответа с соответствующим кодом ошибки и последующей конфигурации контрольных флагов.

При успешном прохождении проверки выполняется нормализация имени и поиск информации в кэше. Если требуемая запись обнаружена, алгоритм анализирует ее актуальность. При истечении времени жизни записи она незамедлительно удаляется из памяти, а процесс поиска продолжается в загруженных зонах, как если бы запись изначально отсутствовала в кэше. Если же запись актуальна, данные копируются в секцию ответа.

Логика поиска в зонах предусматривает несколько вариантов развития событий. Если доменное имя не найдено в хранилище, формируется ответ с кодом ошибки *NXDOMAIN*. При обнаружении совпадения анализируется тип записи. Если найдена запись типа *CNAME*, запускается процесс рекурсивного обхода цепочки псевдонимов для получения конечного адреса. В остальных случаях выполняется прямое добавление записей в структуру ответа. Полученный результат сохраняется в кэш для ускорения обработки последующих аналогичных запросов.

На заключительном этапе производится конфигурирование управляющих флагов, включая установку признака ответа и флагов авторитативности. Завершается работа алгоритма сериализацией структурированных данных в бинарный вид и отправкой сформированного массива байтов клиенту через сетевой интерфейс.

Многопоточная обработка обеспечивается тем, что каждый экземпляр данного алгоритма выполняется в отдельном рабочем потоке, а доступ к общим ресурсам (кэш и хранилище зон) синхронизируется с использованием примитивов взаимного исключения.

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсового проекта был разработан многопоточный сервер *DNS*, реализованный на языке программирования *C++20*. Система обеспечивает полноценную обработку бинарных пакетов в соответствии с *RFC 1035*, включая парсинг запросов и сериализацию ответов. Реализована поддержка основных типов ресурсных записей: *A*, *AAAA* и *CNAME*.

Особое внимание было уделено производительности и безопасности работы в многопоточной среде. Архитектура приложения базируется на использовании пула потоков (*Thread Pool*), что позволяет эффективно распределять нагрузку при обработке одновременных запросов по протоколу *UDP*. Двухуровневая система поиска данных, сочетающая кэш с алгоритмом вытеснения *LRU* и статические файлы зоны, значительно сокращает время отклика сервера. Потокбезопасность доступа к общему кэшу обеспечена применением разделяемого мьютекса (*std::shared_mutex*), реализующего механизм блокировок чтения–записи (*reader–writer lock*).

Разработанная система логирования с поддержкой различных уровней детализации и меток времени в формате *ISO 8601* позволила верифицировать корректность работы сервера в различных сценариях. Тестирование подтвердило стабильность функционирования системы, отсутствие гонок данных и утечек ресурсов при интенсивной нагрузке. Таким образом, поставленные цели по созданию высокопроизводительного и надежного *DNS*–сервера были полностью достигнуты. Спроектированная архитектура продемонстрировала высокую эффективность и масштабируемость.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- [1] Domain Names - Implementation and Specification [Электронный ресурс] / RFC 1035. – Режим доступа: <https://www.rfc-editor.org/rfc/rfc1035>.
- [2] Что такое авторитативный DNS [Электронный ресурс]. – Режим доступа: <https://ngenix.net/learning-center/what-is-authoritative-dns>. – Дата доступа: 23.02.2026.
- [3] Optimizing DNS Performance: TTL Strategies to Reduce Query Count on a DNS Zone [Электронный ресурс]. – Режим доступа: <https://easydns.com/blog/2024/08/12/optimizing-dns-performance-ttl-strategies-to-reduce-query-count-on-a-dns-zone/>. – Дата доступа: 25.03.2026.
- [4] DNS Message Compression Techniques and Limitations [Электронный ресурс]. – Режим доступа: <https://dn.org/dns-message-compression-techniques-and-limitations/>. – Дата доступа: 25.04.2026.
- [5] Negative Caching of DNS Queries (DNS NCACHE) [Электронный ресурс] / RFC 2308. – Режим доступа: <https://www.rfc-editor.org/rfc/rfc2308>.
- [6] LRU, метод вытеснения из кэша [Электронный ресурс]. – Режим доступа: <https://habr.com/ru/articles/136758/>. – Дата доступа: 25.04.2026.
- [7] Многопоточность и Thread Pool в C++ [Электронный ресурс]. – Режим доступа: <https://habr.com/ru/articles/738250/>. – Дата доступа: 25.03.2026.
- [8] Graceful C++ Thread Shutdown [Электронный ресурс]. – Режим доступа: <https://iifx.dev/en/articles/457401731/graceful-c-thread-shutdown-handling-sigint-during-std-thread-join>. – Дата доступа: 26.04.2026.

ПРИЛОЖЕНИЕ А
(Обязательное)
Листинг программного кода

Листинг А.1 – Код файла *dns_packet.cpp*

```
#include "dns_packet.hpp"
#include "dns_record.hpp"
#include <arpa/inet.h>
#include <cstdint>
#include <cstring>
#include <iostream>
#include <string>
#include <utility>

namespace dns {

namespace {

bool read_u16(const uint8_t *data, size_t len, size_t &offset, uint16_t &out) {
    if (offset + 2 > len)
        return false;
    std::memcpy(&out, data + offset, sizeof(uint16_t));
    out = ntohs(out);
    offset += 2;
    return true;
}

bool read_u32(const uint8_t *data, size_t len, size_t &offset, uint32_t &out) {
    if (offset + 4 > len)
        return false;
    std::memcpy(&out, data + offset, sizeof(uint32_t));
    out = ntohl(out);
    offset += 4;
    return true;
}

// Parses a domain name from packet data (supports compression pointers).
// `offset` is advanced only for bytes consumed from the original stream.
bool parse_name(const uint8_t *data, size_t len, size_t &offset,
                std::string &out_name) {
    out_name.clear();

    size_t current = offset;
    bool jumped = false;
    size_t jumps = 0;

    while (true) {
        if (current >= len)
            return false;

        uint8_t length = data[current];

        // In DNS names, if top 2 bits are 11, this is a compression pointer.
        // Compression pointer: 11xxxxxx xxxxxxxx
        if ((length & 0xC0) == 0xC0) {
            if (current + 1 >= len)
                return false;

            uint16_t pointer = static_cast<uint16_t>(((length & 0x3F) << 8) |
                                                    data[current + 1]);
            if (pointer >= len)
                return false;
            jumps++;
            current = pointer;
            continue;
        }

        out_name.append(data, current, length);
        current += length;
    }

    offset = current;
    return true;
}
```

```

        return false;

    if (!jumped) {
        // First pointer consumes 2 bytes in original stream.
        offset = current + 2;
        jumped = true;
    }

    current = pointer;

    // Guard against pointer loops / malformed packets.
    if (++jumps > len)
        return false;
    continue;
}

// Reserved label type 01/10 is invalid for QNAME labels.
if ((length & 0xC0) != 0x00)
    return false;

// End of name.
if (length == 0) {
    if (!jumped)
        offset = current + 1;
    return true;
}

if (length > 63)
    return false;

current += 1;
if (current + length > len)
    return false;

if (!out_name.empty())
    out_name.push_back('.');
out_name.append(reinterpret_cast<const char *>(data + current), length);

current += length;

if (!jumped) {
    // While parsing in original stream (before first jump), keep offset
    // in sync.
    offset = current;
}
}

}

void write_u16(std::vector<uint8_t> &out, uint16_t value) {
    uint16_t be = htons(value);
    const auto *ptr = reinterpret_cast<const uint8_t *>(&be);
    out.insert(out.end(), ptr, ptr + 2);
}

void write_u32(std::vector<uint8_t> &out, uint32_t value) {
    uint32_t be = htonl(value);
    const auto *ptr = reinterpret_cast<const uint8_t *>(&be);
    out.insert(out.end(), ptr, ptr + 4);
}

bool write_name(std::vector<uint8_t> &out, const std::string &name) {
    if (name.empty()) {
        out.push_back(0);
    }
}

```

```

        return true;
    }

    size_t start = 0;
    while (start < name.size()) {
        size_t dot = name.find('.', start);
        size_t end = (dot == std::string::npos) ? name.size() : dot;
        size_t label_len = end - start;

        // Empty label in the middle (e.g. "a..b") is invalid.
        if (label_len == 0)
            return false;
        if (label_len > 63)
            return false;

        out.push_back(static_cast<uint8_t>(label_len));
        out.insert(out.end(), name.begin() + static_cast<std::ptrdiff_t>(start),
                   name.begin() + static_cast<std::ptrdiff_t>(end));

        if (dot == std::string::npos)
            break;
        start = dot + 1;
    }

    // Trailing dot means explicit root label (already covered by terminating
    // zero), so we ignore the final empty label and still emit one root
    // terminator below.
    if (!name.empty() && name.back() == '.') {
        // Reject single "." handled by empty case above, and avoid accepting
        // ".." patterns.
        if (name.size() > 1 && name[name.size() - 2] == '.')
            return false;
    }

    out.push_back(0);
    return true;
}

} // namespace

bool DNSPacket::parse(const uint8_t *data, size_t len) {
    if (data == nullptr || len < 12)
        return false;

    size_t offset = 0;

    if (!read_u16(data, len, offset, header.id))
        return false;
    if (!read_u16(data, len, offset, header.flags))
        return false;
    if (!read_u16(data, len, offset, header.qdcount))
        return false;
    if (!read_u16(data, len, offset, header.ancount))
        return false;
    if (!read_u16(data, len, offset, header.nscount))
        return false;
    if (!read_u16(data, len, offset, header.arcount))
        return false;

    questions.clear();
    questions.reserve(header.qdcount);

    for (uint16_t i = 0; i < header.qdcount; ++i) {

```

```

    Question q;
    if (!parse_name(data, len, offset, q.qname))
        return false;
    if (!read_u16(data, len, offset, q.qtype))
        return false;
    if (!read_u16(data, len, offset, q.qclass))
        return false;
    questions.push_back(std::move(q));
}

answers.clear();
answers.reserve(header.ancount);
for (uint16_t i = 0; i < header.ancount; ++i) {
    ResourceRecord rr;
    if (!parse_name(data, len, offset, rr.name))
        return false;
    if (!read_u16(data, len, offset, rr.type))
        return false;
    if (!read_u16(data, len, offset, rr.rclass))
        return false;
    if (!read_u32(data, len, offset, rr.ttl))
        return false;

    uint16_t rdlength;
    if (!read_u16(data, len, offset, rdlength))
        return false;

    if (offset + rdlength > len)
        return false;

    rr.rdata.assign(data + offset, data + offset + rdlength);
    offset += rdlength;

    answers.push_back(std::move(rr));
}

authorities.clear();
authorities.reserve(header.nscount);
for (uint16_t i = 0; i < header.nscount; ++i) {
    ResourceRecord rr;
    if (!parse_name(data, len, offset, rr.name))
        return false;
    if (!read_u16(data, len, offset, rr.type))
        return false;
    if (!read_u16(data, len, offset, rr.rclass))
        return false;
    if (!read_u32(data, len, offset, rr.ttl))
        return false;

    uint16_t rdlength;
    if (!read_u16(data, len, offset, rdlength))
        return false;

    if (offset + rdlength > len)
        return false;

    rr.rdata.assign(data + offset, data + offset + rdlength);
    offset += rdlength;

    authorities.push_back(std::move(rr));
}

additional.clear();

```

```

        additional.reserve(header.arcount);
        for (uint16_t i = 0; i < header.arcount; ++i) {
            ResourceRecord rr;
            if (!parse_name(data, len, offset, rr.name))
                return false;
            if (!read_u16(data, len, offset, rr.type))
                return false;
            if (!read_u16(data, len, offset, rr.rclass))
                return false;
            if (!read_u32(data, len, offset, rr.ttl))
                return false;

            uint16_t rdlength;
            if (!read_u16(data, len, offset, rdlength))
                return false;

            if (offset + rdlength > len)
                return false;

            rr.rdata.assign(data + offset, data + offset + rdlength);
            offset += rdlength;

            additional.push_back(std::move(rr));
        }

        return true;
    }

std::vector<uint8_t> DNSPacket::serialize() const {
    std::vector<uint8_t> result;

    write_u16(result, header.id);
    write_u16(result, header.flags);
    write_u16(result, static_cast<uint16_t>(questions.size()));
    write_u16(result, static_cast<uint16_t>(answers.size()));
    write_u16(result, static_cast<uint16_t>(authorities.size()));
    write_u16(result, static_cast<uint16_t>(additional.size()));

    for (const auto &q : questions) {
        if (!write_name(result, q.qname)) {
            return {};
        }
        write_u16(result, q.qtype);
        write_u16(result, q.qclass);
    }

    for (const auto &rr : answers) {
        if (!write_name(result, rr.name)) {
            return {};
        }
        write_u16(result, rr.type);
        write_u16(result, rr.rclass);
        write_u32(result, rr.ttl);
        write_u16(result, static_cast<uint16_t>(rr.rdata.size()));
        result.insert(result.end(), rr.rdata.begin(), rr.rdata.end());
    }

    for (const auto &rr : authorities) {
        if (!write_name(result, rr.name)) {
            return {};
        }
        write_u16(result, rr.type);
        write_u16(result, rr.rclass);
    }

```

```

        write_u32(result, rr.ttl);
        write_u16(result, static_cast<uint16_t>(rr.rdata.size()));
        result.insert(result.end(), rr.rdata.begin(), rr.rdata.end());
    }

    for (const auto &rr : additional) {
        if (!write_name(result, rr.name)) {
            return {};
        }
        write_u16(result, rr.type);
        write_u16(result, rr.rclass);
        write_u32(result, rr.ttl);
        write_u16(result, static_cast<uint16_t>(rr.rdata.size()));
        result.insert(result.end(), rr.rdata.begin(), rr.rdata.end());
    }

    return result;
}

} // namespace dns

```

Листинг А.2 – Код файла *resolver.cpp*

```

#include "resolver.hpp"
#include "dns_record.hpp"
#include "logger.hpp"

namespace dns {

// Normalize domain name to FQDN with trailing dot
static std::string normalize_name(const std::string &name) {
    if (name.empty() || name.back() == '.') {
        return name;
    }
    return name + '.';
}

Resolver::Resolver(ZoneLoader &zone_loader, DNSCache &cache)
    : zone_loader_(zone_loader), cache_(cache) {}

DNSPacket Resolver::resolve(const DNSPacket &query) {
    DNSPacket response;
    response.header.id = query.header.id;

    if (query.questions.empty()) {
        set_response_flags(response.header, false, 2);
        return response;
    }

    const auto &question = query.questions[0];
    response.questions.push_back(question);
    response.header.qdcount = 1;

    std::string qname = normalize_name(question.qname);
    RecordType query_type = static_cast<RecordType>(question.qtype);

    // Check Cache first
    auto cached_records = cache_.get(qname, question.qtype);
    if (cached_records.has_value()) {
        response.answers = std::move(*cached_records);
        response.header.ancount =
            static_cast<uint16_t>(response.answers.size());
        set_response_flags(

```

```

        response.header, false,
        0); // Cached responses are generally non-authoritative
    return response;
}

// Cache Miss, proceed with normal resolution
if (query_type == RecordType::A || query_type == RecordType::AAAA) {
    if (follow_cname_chain(qname, query_type, response.answers)) {
        set_response_flags(response.header, true, 0);
        response.header.ancount =
            static_cast<uint16_t>(response.answers.size());

        // Assuming default TTL of 60 for resolved records without explicit
        // TTL logic
        uint32_t min_ttl = 60;
        if (!response.answers.empty()) {
            min_ttl = response.answers[0].ttl;
            for (const auto &ans : response.answers) {
                if (ans.ttl < min_ttl)
                    min_ttl = ans.ttl;
            }
        }
        cache_.put(qname, question.qtype, response.answers, min_ttl);
    } else {
        set_response_flags(response.header, true, 3);
    }
} else {
    auto records = zone_loader_.get_records(qname, query_type);
    if (!records.empty()) {
        response.answers = std::move(records);
        response.header.ancount =
            static_cast<uint16_t>(response.answers.size());
        set_response_flags(response.header, true, 0);

        uint32_t min_ttl = 60;
        if (!response.answers.empty()) {
            min_ttl = response.answers[0].ttl;
            for (const auto &ans : response.answers) {
                if (ans.ttl < min_ttl)
                    min_ttl = ans.ttl;
            }
        }
        cache_.put(qname, question.qtype, response.answers, min_ttl);
    } else {
        set_response_flags(response.header, true, 3);
    }
}

return response;
}

bool Resolver::follow_cname_chain(
    const std::string &start_name, RecordType target_type,
    std::vector<DNSPacket::ResourceRecord> &answer_records, int depth) {
    if (depth >= MAX_CNAME_DEPTH) {
        LOG_WARNING("CNAME chain too deep, possible loop detected for "
            << start_name);
        return false;
    }

    std::string search_name = normalize_name(start_name);

    auto direct_records = zone_loader_.get_records(search_name, target_type);

```

```

        if (!direct_records.empty()) {
            answer_records.insert(answer_records.end(), direct_records.begin(),
                                  direct_records.end());
            return true;
        }

        auto cname_records =
            zone_loader_.get_records(search_name, RecordType::CNAME);

        if (cname_records.empty()) {
            return false;
        }

        for (const auto &cname_rec : cname_records) {
            answer_records.push_back(cname_rec);

            auto parsed = parse_rdata(RecordType::CNAME, cname_rec.rdata.data(),
                                       cname_rec.rdata.size());

            if (!parsed) {
                LOG_ERROR("Failed to parse CNAME rdata");
                return false;
            }

            auto *cname = dynamic_cast<CNAMERecord *>(parsed.get());
            if (!cname) {
                LOG_ERROR("CNAME record type mismatch");
                return false;
            }

            std::string target_name = normalize_name(cname->cname());

            if (follow_cname_chain(target_name, target_type, answer_records,
                                   depth + 1)) {
                return true;
            }
        }

        return false;
    }

void Resolver::set_response_flags(DNSPacket::Header &header, bool
authoritative, uint8_t rcode) {
    uint16_t flags = 0;
    flags |= (1 << 15);

    if (authoritative) {
        flags |= (1 << 10);
    }

    flags |= (rcode & 0x0F);

    header.flags = flags;
}

} // namespace dns

```

Листинг А.3 – Код файла *cache.cpp*

```

#include "cache.hpp"
#include "logger.hpp"
#include <mutex>

namespace dns {

```

```

DNSCache::DNSCache(size_t max_size) : max_size_(max_size > 0 ? max_size : 1) {}

void DNSCache::put(const std::string &name, uint16_t type,
                  const std::vector<DNSPacket::ResourceRecord> &records,
                  uint32_t ttl) {
    if (records.empty()) {
        return;
    }

    CacheKey key{name, type};
    CacheValue value{records, std::chrono::steady_clock::now(), ttl};

    std::lock_guard<std::mutex> lock(mutex_);

    auto it = cache_map_.find(key);
    if (it != cache_map_.end()) {
        // Update existing item and move to front
        cache_list_.erase(it->second);
    } else if (cache_map_.size() >= max_size_) {
        // Evict least recently used
        cache_map_.erase(cache_list_.back().first);
        cache_list_.pop_back();
    }

    cache_list_.emplace_front(std::make_pair(key, value));
    cache_map_[key] = cache_list_.begin();

    LOG_INFO("Cache PUT: " << name << " Type: " << type << " TTL: " << ttl);
}

std::optional<std::vector<DNSPacket::ResourceRecord>>
DNSCache::get(const std::string &name, uint16_t type) {
    CacheKey key{name, type};
    std::lock_guard<std::mutex> lock(mutex_);

    auto it = cache_map_.find(key);
    if (it != cache_map_.end()) {
        auto now = std::chrono::steady_clock::now();
        auto duration = std::chrono::duration_cast<std::chrono::seconds>(
            now - it->second->second.timestamp()
            .count());

        if (duration <= it->second->second.ttl) {
            auto records = it->second->second.records;
            hits_.fetch_add(1, std::memory_order_relaxed);

            // Promote to front of LRU (most recently used)
            cache_list_.splice(cache_list_.begin(), cache_list_, it->second);

            LOG_DEBUG("Cache HIT: " << name << " Type: " << type);
            return records;
        } else {
            // Expired, so we should clean it up
            cache_list_.erase(it->second);
            cache_map_.erase(it);
            LOG_DEBUG("Cache EXPIRED: " << name << " Type: " << type);
        }
    }

    misses_.fetch_add(1, std::memory_order_relaxed);
    LOG_DEBUG("Cache MISS: " << name << " Type: " << type);
    return std::nullopt;
}

```

```

}

size_t DNSCache::get_hits() const {
    return hits_.load(std::memory_order_relaxed);
}

size_t DNSCache::get_misses() const {
    return misses_.load(std::memory_order_relaxed);
}

size_t DNSCache::size() const {
    std::lock_guard<std::mutex> lock(mutex_);
    return cache_map_.size();
}

void DNSCache::clear() {
    std::lock_guard<std::mutex> lock(mutex_);
    cache_map_.clear();
    cache_list_.clear();
    hits_.store(0, std::memory_order_relaxed);
    misses_.store(0, std::memory_order_relaxed);
}

} // namespace dns

```

ПРИЛОЖЕНИЕ Б
(Обязательное)
Функциональная схема обработки запросов и кэширования

ПРИЛОЖЕНИЕ В
(Обязательное)
Блок-схема алгоритма поиска и формирования DNS-пакета ответа

ПРИЛОЖЕНИЕ Г
(Обязательное)
Графический интерфейс пользователя

ПРИЛОЖЕНИЕ Д
(Обязательное)
Ведомость курсового проекта